

Final Algorithm Lab Project

by:

Ahmed Mostafa Hussein

Description of code:

Find smallest number of two arrays

You are given two unsorted arrays of integers a and b of the same sizes, sort them using two different sorting (Merge Sort & Quick Sort) algorithms then put the smallest number in the same index in third array c (Note: The numbers should be different in array a and b). Then Search about number in array c and output the number is found in index i in array a or b. Print not Found if the number is not exist in both arrays. Then calculate Big (O) of your program.

Input

- The second line contains n integers — the elements of array a.
- The third line contains m integers — the elements of array b.
- Search in the output array.

Output

- Array c which contains the smallest number between array a and b in the same index.
- Result of searching (Found at any index in which array /or Not Found).

Run of code without GUI:

```
Output - Run (FinalAlgorithmProject)
-----<"C:\com.mycompany\FinalAlgorithmProject">-----
Building FinalAlgorithmProject 1.0-SNAPSHOT
from pom.xml
-----[ jar ]-----
--- resources:3.3.1:resources (default-resources) @ FinalAlgorithmProject ---
skip non existing resourceDirectory E:\aBooks of College\second year second term\an
--- compiler:3.11.0:compile (default-compile) @ FinalAlgorithmProject ---
Changes detected - recompiling the module! :source
Compiling 1 source file with javac [debug target 21] to target\classes
--- exec:3.1.0:exec (default-cli) @ FinalAlgorithmProject ---
Enter elements of array a separated by spaces:
3 2 5 1 7
Enter elements of array b separated by spaces (must same size array a):
8 4 6 10 9
Array c: [1, 2, 3, 5, 7]
Enter Search Key:
3
3 is found at index 2 in array a

The time complexity of Merge Sort is O(n log n)
The time complexity of Quick Sort is O(n^2)
The time complexity of the binarySearch function is O(log n)
The Big O complexity of the entire program is O(n^2)
-----
BUILD SUCCESS
-----
Total time: 41.351 s
Finished at: 2024-05-11T16:59:32+03:00
-----
```

Run of code with GUI:

Final Algorithm Project

Array a:

8 4 2 6 9

Array b:

10 5 3 1 7

Search Key:

9

Array c: [1, 3, 5, 7, 9]

9 is found at index 4 in array a

The time complexity of Merge Sort is O(n log n)

The time complexity of Quick Sort is O(n^2)

The time complexity of the binarySearch function is O(log n)

The Big O complexity of the entire program is O(n^2)

Search

Code of program without GUI:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
 * license
 */

package com.mycompany.finalalgorithmproject;

/**
 *
 * @author Ahmed mostafa
 */
import java.util.Arrays;
import java.util.Scanner;

public class FinalAlgorithmProject {

    // Merge Sort implementation
    public static void mergeSort(int[] arr) {
        if (arr.length <= 1) return;

        int mid = arr.length / 2;
        int[] left = Arrays.copyOfRange(arr, 0, mid);
        int[] right = Arrays.copyOfRange(arr, mid, arr.length);

        mergeSort(left);
        mergeSort(right);
    }
}
```

```
merge(arr, left, right);  
}
```

```
private static void merge(int[] arr, int[] left, int[] right) {  
    int i = 0, j = 0, k = 0;  
    while (i < left.length && j < right.length) {  
        if (left[i] <= right[j]) {  
            arr[k++] = left[i++];  
        } else {  
            arr[k++] = right[j++];  
        }  
    }  
    while (i < left.length) {  
        arr[k++] = left[i++];  
    }  
    while (j < right.length) {  
        arr[k++] = right[j++];  
    }  
}
```

// Quick Sort implementation

```
public static void quickSort(int[] arr, int low, int high) {  
    if (low < high) {  
        int pi = partition(arr, low, high);  
  
        quickSort(arr, low, pi - 1);  
        quickSort(arr, pi + 1, high);  
    }  
}
```

```
}
```

```
private static int partition(int[] arr, int low, int high) {
```

```
    int pivot = arr[high];
```

```
    int i = (low - 1);
```

```
    for (int j = low; j < high; j++) {
```

```
        if (arr[j] < pivot) {
```

```
            i++;
```

```
            int temp = arr[i];
```

```
            arr[i] = arr[j];
```

```
            arr[j] = temp;
```

```
        }
```

```
    }
```

```
    int temp = arr[i + 1];
```

```
    arr[i + 1] = arr[high];
```

```
    arr[high] = temp;
```

```
    return i + 1;
```

```
}
```

```
// binarySearch function
```

```
public static int binarySearch(int[] arr, int key) {
```

```
    int left = 0;
```

```
    int right = arr.length - 1;
```

```
    while (left <= right) {
```

```
        int mid = left + (right - left) / 2;
```

```

// Check if key is present at mid
if (arr[mid] == key)
    return mid;

// If key is greater, ignore left half
if (arr[mid] < key)
    left = mid + 1;

// If key is smaller, ignore right half
else
    right = mid - 1;
}

// If the element is not present in the array
return -1;
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    // Input array a
    System.out.println("Enter elements of array a separated by spaces:");
    String[] aInput = scanner.nextLine().split("\\s+");
    int[] a = Arrays.stream(aInput).mapToInt(Integer::parseInt).toArray();

    // Input array b
    System.out.println("Enter elements of array b separated by spaces (must same size array a):");
    String[] bInput = scanner.nextLine().split("\\s+");

```

```

int[] b = Arrays.stream(bInput).mapToInt(Integer::parseInt).toArray();

// Sort arrays a and b
mergeSort(a);
quickSort(b, 0, b.length - 1);

// Populate array c with smallest elements
int[] c = new int[a.length];
for (int i = 0; i < a.length; i++) {
    c[i] = Math.min(a[i], b[i]);
}

System.out.println("Array c: " + Arrays.toString(c));

// Search key in array c
System.out.println("Enter Search Key: ");
int key = scanner.nextInt();
int index = binarySearch(c, key);

if (index != -1) {
    if (index < a.length) {
        System.out.println(key + " is found at index " + index + " in array a");
    } else if (index - a.length < b.length) {
        System.out.println(key + " is found at index " + (index - a.length) + " in array b");
    } else {
        System.out.println("Key found at an invalid index.");
    }
} else {

```

```
        System.out.println("Not Found");
    }
    scanner.close();

    System.out.println("\nThe time complexity of Merge Sort is O(n log n)");
    System.out.println("The time complexity of Quick Sort is O(n^2)");
    System.out.println("The time complexity of the binarySearch function is O(log n)");
    System.out.println("The Big O complexity of the entire program is O(n^2)");
}
}
```


Code of program with GUI:

```
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this
 * license
 */

package com.mycompany.finalalgorithmprojectgui;

/**
 *
 * @author Ahmed mostafa
 */
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.util.Arrays;

public class FinalAlgorithmProjectGUI extends JFrame {

    private final JTextField arrayATextField;
    private final JTextField arrayBTextField;
    private final JTextField searchKeyTextField;
    private final JTextArea resultTextArea;

    public FinalAlgorithmProjectGUI() {
        setTitle("Final Algorithm Project");
        setSize(400, 400);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }
}
```

```
setLayout(new BorderLayout());
```

```
JPanel inputPanel = new JPanel(new GridLayout(4, 2));
```

```
inputPanel.add(new JLabel("Array a:"));
```

```
arrayATextField = new JTextField();
```

```
inputPanel.add(arrayATextField);
```

```
inputPanel.add(new JLabel("Array b:"));
```

```
arrayBTextField = new JTextField();
```

```
inputPanel.add(arrayBTextField);
```

```
inputPanel.add(new JLabel("Search Key:"));
```

```
searchKeyTextField = new JTextField();
```

```
inputPanel.add(searchKeyTextField);
```

```
add(inputPanel, BorderLayout.NORTH);
```

```
JButton searchButton = new JButton("Search");
```

```
resultTextArea = new JTextArea();
```

```
resultTextArea.setEditable(false);
```

```
JScrollPane resultScrollPane = new JScrollPane(resultTextArea);
```

```
add(resultScrollPane, BorderLayout.CENTER);
```

```
JPanel arrayCPanel = new JPanel(new BorderLayout());
```

```
JTextArea arrayCTextArea = new JTextArea();
```

```
arrayCTextArea.setEditable(false);
```

```
arrayCPanel.add(new JLabel("Array c:"), BorderLayout.NORTH);
```

```

arrayCPanel.add(new JScrollPane(arrayCTextArea), BorderLayout.CENTER);
add(arrayCPanel, BorderLayout.SOUTH);

add(searchButton, BorderLayout.SOUTH);

searchButton.addActionListener((ActionEvent e) -> {
    searchButtonClicked();
});
}

private void searchButtonClicked() {
    String arrayAInput = arrayATextField.getText();
    String arrayBInput = arrayBTextField.getText();
    String searchKeyInput = searchKeyTextField.getText();

    int[] a = Arrays.stream(arrayAInput.split("\\s+")).mapToInt(Integer::parseInt).toArray();
    int[] b = Arrays.stream(arrayBInput.split("\\s+")).mapToInt(Integer::parseInt).toArray();
    int key = Integer.parseInt(searchKeyInput);

    int[] c = new int[a.length];
    mergeSort(a);
    quickSort(b, 0, b.length - 1);
    for (int i = 0; i < a.length; i++) {
        c[i] = Math.min(a[i], b[i]);
    }

    StringBuilder cStringBuilder = new StringBuilder();
    cStringBuilder.append("[ ");

```

```

for (int i = 0; i < c.length; i++) {
    cStringBuilder.append(c[i]);
    if (i < c.length - 1) {
        cStringBuilder.append(", ");
    }
}
cStringBuilder.append(" ]");
resultTextArea.setText("");
resultTextArea.append("Array c: " + cStringBuilder.toString() + "\n");

int index = binarySearch(c, key);

if (index != -1) {
    if (index < a.length) {
        resultTextArea.append(key + " is found at index " + index + " in array a\n");
    } else {
        resultTextArea.append(key + " is found at index " + (index - a.length) + " in array b\n");
    }
} else {
    resultTextArea.append("Not Found\n");
}

resultTextArea.append("\nThe time complexity of Merge Sort is O(n log n)");
resultTextArea.append("\nThe time complexity of Quick Sort is O(n^2)");
resultTextArea.append("\nThe time complexity of the binarySearch function is O(log n)");
resultTextArea.append("\nThe Big O complexity of the entire program is O(n^2)");
}

```

```
// Merge Sort implementation

public static void mergeSort(int[] arr) {
    if (arr.length <= 1) return;

    int mid = arr.length / 2;
    int[] left = Arrays.copyOfRange(arr, 0, mid);
    int[] right = Arrays.copyOfRange(arr, mid, arr.length);

    mergeSort(left);
    mergeSort(right);

    merge(arr, left, right);
}
```

```
private static void merge(int[] arr, int[] left, int[] right) {
    int i = 0, j = 0, k = 0;
    while (i < left.length && j < right.length) {
        if (left[i] <= right[j]) {
            arr[k++] = left[i++];
        } else {
            arr[k++] = right[j++];
        }
    }
    while (i < left.length) {
        arr[k++] = left[i++];
    }
    while (j < right.length) {
        arr[k++] = right[j++];
    }
}
```

```
    }  
}
```

// Quick Sort implementation

```
public static void quickSort(int[] arr, int low, int high) {  
    if (low < high) {  
        int pi = partition(arr, low, high);  
  
        quickSort(arr, low, pi - 1);  
        quickSort(arr, pi + 1, high);  
    }  
}
```

```
private static int partition(int[] arr, int low, int high) {  
    int pivot = arr[high];  
    int i = (low - 1);  
    for (int j = low; j < high; j++) {  
        if (arr[j] < pivot) {  
            i++;  
            int temp = arr[i];  
            arr[i] = arr[j];  
            arr[j] = temp;  
        }  
    }  
    int temp = arr[i + 1];  
    arr[i + 1] = arr[high];  
    arr[high] = temp;
```

```
        return i + 1;
    }

    // Search function
    public static int binarySearch(int[] arr, int key) {
        int left = 0;
        int right = arr.length - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;

            // Check if key is present at mid
            if (arr[mid] == key)
                return mid;

            // If key is greater, ignore left half
            if (arr[mid] < key)
                left = mid + 1;

            // If key is smaller, ignore right half
            else
                right = mid - 1;
        }

        // If the element is not present in the array
        return -1;
    }
```

```
public static void main(String[] args) {  
    SwingUtilities.invokeLater(() -> {  
        FinalAlgorithmProjectGUI gui = new FinalAlgorithmProjectGUI();  
        gui.setVisible(true);  
    });  
}  
}
```